

1. TITLE PAGE

LegalEase: AI-Powered Legal Document Generator

Sub-Title: Logic in routes.py

Naan Mudhalvan Project Report

An intelligent web-based application for generating structured legal documents with the help of Artificial Intelligence.

Project Domain: Artificial Intelligence / Web Application / Legal Technology

Academic Year: 2026–2027

2. ABSTRACT

LegalEase is an AI-powered legal document generation system designed to simplify the preparation of common legal documents. The application provides a structured interface where users can enter relevant information and receive a properly organized document draft.

The central project logic is implemented through the **routes.py** module. It manages application routes, receives user inputs, validates the request, connects the interface with the document-generation workflow, and returns the generated result to the user.

The system aims to reduce repetitive manual formatting, improve consistency, and make document preparation easier for students, small organizations, and general users. LegalEase is intended as a document drafting assistant and does not replace professional legal advice.

3. INTRODUCTION

Legal documents often require a clear structure, correct terminology, consistent formatting, and complete information. Preparing such documents manually can be time-consuming, particularly when users are unfamiliar with standard document formats.

LegalEase addresses this problem through a web-based application that combines a simple user interface with AI-assisted content generation. Users provide the required information and select the type of document they need. The backend processes the request and generates a structured draft.

The project demonstrates how Artificial Intelligence can be integrated into a practical software application while maintaining a clear separation between user interface, routing logic, processing, and output generation.

4. PROBLEM STATEMENT

Traditional legal document preparation may involve searching for templates, copying information, editing multiple sections, checking formatting, and repeatedly correcting mistakes. These activities can create unnecessary effort and inconsistent outputs.

Problem: Develop a user-friendly system that can collect required information and generate a structured legal-document draft automatically.

Need: The application should provide clear routes, input handling, validation, document generation, error handling, and a readable final output.

5. OBJECTIVES

Primary objective: To develop an AI-powered web application for generating structured legal document drafts.

Supporting objectives:

- Provide a simple interface for entering document information.
- Use routes.py as the central controller for application requests.
- Validate required fields before document generation.
- Connect user input with the AI/document-generation workflow.
- Produce consistent and readable document output.
- Handle invalid input and application errors safely.
- Create a modular design that can be extended with additional document types.

6. SCOPE OF THE PROJECT

LegalEase can be designed to support multiple categories of document drafting, such as basic agreements, notices, declarations, applications, and other structured templates, depending on the implementation.

The scope includes front-end input collection, backend routing, validation, AI-assisted text generation, formatting, and output presentation. The project focuses on document drafting assistance rather than legal representation or legal decision-making.

Future versions may include user accounts, document history, PDF export, multilingual support, template management, stronger validation, and integration with approved legal databases.

7. SYSTEM REQUIREMENTS

Hardware: A standard computer or laptop with internet/network access, sufficient RAM for the development environment, and normal storage capacity.

Software: Python 3.x, a Python web framework such as Flask, an IDE or code editor, a web browser, required Python packages, and an AI/API service where applicable.

Development environment: The project can be organized into separate files for routes, templates, static resources, configuration, and document-generation logic.

8. SYSTEM ARCHITECTURE

The system follows a simple request-processing architecture. The user interacts with the web page, submits document details, and the request is received by the backend route.

Flow: User → Web Interface → routes.py → Input Validation → AI/Document Logic → Generated Draft → Web Response.

This structure makes the application easier to maintain because routing decisions are separated from presentation. The routes.py module acts as the bridge between the browser and the processing components.

9. TECHNOLOGIES USED

Python: Used for backend application logic and processing.

Flask: A lightweight Python web framework suitable for routing and server-side application development.

HTML/CSS: Used to create and style the user interface.

JavaScript: Can be used for client-side interactions and form behavior.

AI service/model: Used where configured to generate or assist with document text.

Jinja templates: Used to render dynamic information in web pages.

10. PROJECT MODULES

- 1. User Interface Module:** Collects document type and user-provided information.
- 2. Routing Module:** Handles URLs and HTTP requests.
- 3. Validation Module:** Checks whether required fields contain acceptable values.
- 4. AI Generation Module:** Creates or assists with the requested draft.
- 5. Output Module:** Displays or prepares the generated document.
- 6. Error Handling Module:** Provides safe responses for invalid input or service failures.
- 7. Configuration Module:** Stores application settings and environment-specific values.

11. LOGIC IN routes.py

routes.py is the core routing layer of the project. It defines what the application should do when a user visits a URL or submits a form.

A typical route receives an HTTP request, reads form or JSON data, validates the information, calls the required processing function, and returns a template or generated response.

Conceptual flow: Request → Route Function → Read Input → Validate → Process/Generate → Prepare Result → Return Response.

The route should avoid unnecessary business logic when possible. Keeping AI generation and document formatting in separate functions improves readability, testing, and maintenance.

12. routes.py – REQUEST HANDLING

When the user opens the application, a route can render the home page. When the user submits the document form, a POST route receives the data.

Example conceptual logic:

```
if request.method == 'POST':  
    data = request.form  
    validate(data)  
    result = generate_document(data)  
    return render_template('result.html', result=result)
```

The exact implementation depends on the project's Flask structure. The important principle is that routes.py coordinates the request rather than mixing every application responsibility into one large function.

13. INPUT VALIDATION & ERROR HANDLING

Validation is important because incomplete or incorrect information can produce poor document drafts. Required fields should be checked before the generation process begins.

Examples include checking whether a document type is selected, required names are supplied, dates follow an expected format, and text fields are not unexpectedly empty.

Error handling should return a clear message instead of exposing internal exceptions. API keys and confidential configuration values should be stored securely in environment variables rather than hard-coded into routes.py.

14. AI DOCUMENT GENERATION WORKFLOW

After validation, the application can construct a structured prompt or template using the user-provided information. The AI component processes the request and returns a draft.

The application should then apply formatting and present the output clearly. Where possible, generated content should be reviewed for completeness, consistency, and suitability before being treated as a final document.

Important: AI-generated legal text may contain errors or omissions. LegalEase should clearly communicate that the generated output is a drafting aid and that users should obtain appropriate professional legal review when necessary.

15. USER INTERFACE & USER FLOW

Step 1: Open the LegalEase application.

Step 2: Select the required document category.

Step 3: Enter the requested information.

Step 4: Submit the form.

Step 5: routes.py receives and validates the request.

Step 6: The generation workflow creates the document draft.

Step 7: The result page displays the generated content.

The interface should use clear labels, helpful placeholders, validation messages, and a simple navigation structure.

16. TESTING

Testing verifies that each important part of LegalEase behaves as expected.

Functional testing: Verify that each route opens correctly and that valid form submissions produce results.

Validation testing: Submit empty or invalid fields and confirm that useful error messages appear.

Integration testing: Verify communication between routes.py and the generation component.

UI testing: Check that pages display correctly on common screen sizes.

Error testing: Simulate unavailable services and confirm that the application fails gracefully.

17. ADVANTAGES & LIMITATIONS

Advantages: Faster drafting workflow, consistent structure, easy-to-use interface, modular architecture, reduced repetitive formatting, and potential for multiple document templates.

Limitations: AI output can contain factual or drafting errors; document quality depends on input quality and configured generation services; legal requirements can vary by jurisdiction and document type.

Therefore, LegalEase should be positioned as an assistance and drafting tool rather than a substitute for a qualified legal professional.

18. FUTURE ENHANCEMENTS

Future development can add downloadable PDF/DOCX generation, user authentication, document history, multilingual Tamil/English support, reusable templates, stronger field validation, version tracking, audit logs, and role-based access.

Other enhancements may include a document preview before generation, configurable clauses, citation/reference support, accessibility improvements, and deployment using a secure cloud environment.

A carefully designed feedback system can also help developers identify recurring generation issues and improve templates and validation rules.

19. CONCLUSION

LegalEase demonstrates how Artificial Intelligence and web technologies can be combined to simplify structured document drafting. The project provides a practical example of applying software engineering concepts such as routing, validation, modular design, error handling, and AI integration.

The **routes.py** module plays an important role by connecting user requests with the application's processing workflow. A clean routing design makes the project easier to understand, test, maintain, and extend.

With appropriate legal review, security controls, and further development, the project can be expanded into a more capable legal-document assistance platform.

20. PROJECT SUMMARY / VIVA POINTS

Project Title: LegalEase: AI-Powered Legal Document Generator

Sub-title: Logic in routes.py

Core idea: Generate structured legal document drafts using a web application and AI-assisted workflow.

Key file: routes.py — handles application routes, requests, validation flow, and responses.

Main technologies: Python, Flask, HTML, CSS, JavaScript, Jinja, and an AI service/model where configured.

Key benefit: Simplifies repetitive document drafting and formatting.

Important safety point: Generated text should be reviewed and is not a substitute for professional legal advice.

Final statement: LegalEase is a practical Naan Mudhalvan project demonstrating the application of AI, Python web development, and modular backend routing to a real-world documentation problem.